

Documentação Técnica — Portal do Cliente One Finance

Versão: 2.0.0 Data: Julho de 2026

1. Visão geral

O Portal do Cliente One Finance é uma aplicação web estática (HTML/CSS/JS puro, sem build step) hospedada no GitHub Pages, que oferece a clientes da One Finance BPO Financeiro acesso a dados financeiros integrados via Conta Azul e armazenados em um banco Neon (Postgres).

A aplicação tem dois perfis de uso:

- **Administradores** — gerenciam clientes, perfis de usuário e sincronização de dados.
- **Usuários limitados** — acessam apenas os módulos financeiros das empresas às quais têm acesso liberado.

1.1 Módulos

Módulo	Arquivo	Função
Login	index.html	Autenticação do usuário
Recuperar senha	recuperar-senha.html	Geração de link de redefinição (token de 30 min)
Nova senha	nova-senha.html	Validação do token e atualização do hash
Gerenciamento	gerenciamento.html	CRUD de clientes e perfis, log de atividades, sincronização
Dashboard	dashboard.html	Seleção de módulo por cliente
Contas a Receber	contas-receber.html	Recebíveis, vencimentos, status
Contas a Pagar	contas-pagar.html	Pagamentos, vencimentos, categorias
Fluxo de Caixa	fluxo-caixa.html	Entradas, saídas, saldo do período
DRE	dre.html	Demonstrativo de Resultado Gerencial
Perfil	perfil.html	Dados da sessão, troca de senha
Privacidade	privacidade.html	Política de privacidade (LGPD)

2. Arquitetura

2.1 Componentes

- **Frontend** — HTML/CSS/JS estático, servido via GitHub Pages, sem etapa de build.
- **Armazenamento de dados de configuração** — arquivos `data/clients.json` e `data/perfis.json` versionados no mesmo repositório GitHub (`OneFinanceBPO/Portal-do-Cliente`).
- **Cloudflare Worker** (`cloudflare-worker.js`) — proxy que expõe duas rotas:
 - `POST /api/sync` — grava JSON no repositório GitHub via API, usando um token que nunca é exposto ao navegador.
 - `GET /api/financeiro` — consulta o Neon Postgres (tabela `extrato_movimentacoes`) e devolve dados agregados por mês, categoria e pendências.
- **Neon (Postgres)** — fonte dos dados financeiros reais (extrato de movimentações por empresa/CNPJ), alimentado externamente via integração com Conta Azul.

2.2 Fluxo de dados

O navegador acessa o Cloudflare Worker para as duas operações protegidas (escrita no GitHub e leitura financeira do Neon). O Worker valida um cabeçalho `X-App-Secret` compartilhado antes de processar qualquer requisição.

Importante: a leitura de `perfis.json` e `clients.json` para fins de **login e recuperação de senha** não passa pelo Worker — é feita por `fetch` direto em `raw.githubusercontent.com`, sem autenticação. Ver seção 5.1 para as implicações disso.

2.3 Modo legado

Para desenvolvimento, existe um modo alternativo em que um token do GitHub com permissão de escrita é armazenado diretamente no `localStorage` do navegador, dispensando o Worker. O próprio `SETUP.md` do projeto alerta que esse modo não deve ser usado em produção.

3. Autenticação e controle de acesso

3.1 Login

1. O usuário informa e-mail e senha em `index.html`.
2. O script busca `perfis.json` do GitHub (fallback para `localStorage` em caso de erro de rede).

3. A senha informada é hasheada com SHA-256 (`crypto.subtle.digest`) e comparada ao hash salvo no perfil (com suporte a senha em texto puro para compatibilidade retroativa).
4. Em caso de sucesso, os dados da sessão são salvos em `sessionStorage` (`AUTH.setSession`) e o usuário é redirecionado para `gerenciamento.html`.
5. Existe um bypass fixo para um e-mail de administrador definido em código (`ADMIN_EMAILS`).

3.2 Recuperação de senha

1. `recuperar-senha.html` gera um token aleatório de 32 bytes e uma expiração de 30 minutos, salvos tanto em `perfis.json` (sincronizado ao GitHub) quanto em `localStorage`.
2. O link de redefinição (`nova-senha.html?email=...&token=...`) é exibido para o operador copiar e enviar manualmente ao usuário — não há envio automático de e-mail.
3. `nova-senha.html` valida o token (local ou remoto) e, se válido, grava o novo hash SHA-256 no `perfis.json`.

3.3 Controle de tipo de usuário (admin / limitado)

A distinção entre administrador e usuário limitado é feita **inteiramente no frontend**, em `gerenciamento.html`:

```
js
const isRawAdmin = !loginEmail || loginEmail.includes('adm') || loginEmail.incl
```

Se não for um "admin bruto" por e-mail, o sistema busca o perfil correspondente em `perfis.json` e verifica o campo `tipo` (`adm` ou `limitado`) e a lista `acesso` (IDs de clientes liberados) para decidir o que exibir.

3.4 Sessão

- Sessão armazenada em `sessionStorage` (limpa ao fechar a aba).
- Timeout automático por inatividade de 20 minutos, com aviso em contagem regressiva antes do logout.
- Todo login, logout e evento de segurança é registrado no log de atividades local.

4. Log de atividades

`js/activity-log.js` registra login, logout, CRUD de clientes e perfis, eventos de sincronização e configuração de token, com filtro por categoria (login, clientes, perfis,

sincronização, segurança) e exportação em CSV. O log é armazenado localmente (conforme a política de privacidade, por até 90 dias).

5. Achados de segurança

Esta seção resume os pontos identificados na análise do código-fonte. Está organizada por severidade.

5.1 Crítico — exposição de dados de autenticação

O login busca `perfis.json` via `raw.githubusercontent.com` **sem token de autenticação**. Para isso funcionar no navegador, o repositório precisa ser público — o que contradiz a afirmação de "repositório privado" feita em `privacidade.html`. Se o repositório for público, qualquer pessoa pode baixar `perfis.json` e obter:

- todos os e-mails cadastrados;
- os hashes SHA-256 das senhas;
- tokens de recuperação de senha ativos (`resetToken`/`resetExpiry`), permitindo sequestro de conta dentro da janela de 30 minutos.

Ação recomendada: confirmar a visibilidade do repositório no GitHub imediatamente. Se público, tornar privado e mover a leitura de perfis para uma rota autenticada no Worker.

5.2 Crítico — controle de acesso decorativo

A verificação de admin/usuário limitado roda inteiramente no navegador, a partir de `localStorage.getItem('of_login_email')`. Qualquer usuário com acesso ao console do navegador pode alterar esse valor e obter privilégios de administrador, incluindo visualização de todos os clientes e edição de perfis — sem que exista qualquer validação server-side equivalente.

Ação recomendada: mover a decisão de permissão para o Cloudflare Worker (o Worker deve conhecer o usuário autenticado e aplicar as regras, não confiar em dado enviado pelo cliente).

5.3 Alto — hash de senha fraco

As senhas usam SHA-256 simples, sem *salt* nem custo computacional (não é bcrypt/scrypt/Argon2/PBKDF2). Combinado com o item 5.1, isso é vulnerável a ataques de força bruta/rainbow table caso os hashes vazem.

5.4 Alto — escrita de arquivo sem validação de caminho no Worker

A rota `POST /api/sync` aceita um campo `path` livre no corpo da requisição e o usa diretamente para montar a URL da API do GitHub, sem restringir a uma lista de arquivos permitidos. Quem possuir o `APP_SECRET` pode escrever em qualquer arquivo do repositório.

Ação recomendada: validar `path` contra uma lista fixa (`data/clients.json`, `data/perfis.json`).

5.5 Médio — CORS permissivo por padrão

`ALLOWED_ORIGIN` cai para `'*'` quando a variável de ambiente não está definida no Worker.

5.6 Médio — sem limite de tentativas no Worker

A validação do `APP_SECRET` é uma comparação simples de string, sem *rate limiting*, o que facilita ataques de força bruta contra o segredo.

5.7 A verificar — endpoint HTTP do Neon

O Worker monta o endpoint de consulta ao Neon como `https://{host}/sql` a partir da connection string. Essa não é a forma documentada oficialmente pela Neon para a API HTTP — vale confirmar em produção se a consulta realmente funciona ou se é necessário usar a biblioteca oficial (`@neondatabase/serverless`).

6. Pontos positivos

- Separação do token do GitHub em um proxy (Worker), evitando exposição direta no navegador — desde que a validação de `path` seja corrigida.
 - Sessão em `sessionStorage` (não persistente) com timeout de inatividade.
 - Consultas ao Neon parametrizadas (`{1}`, `{2}`), protegidas contra SQL injection.
 - Política de privacidade completa e estruturada conforme a LGPD.
 - UX cuidada: tour de onboarding, toasts, log de atividades, busca com destaque de termo, layout responsivo.
-

7. Recomendações priorizadas

1. Confirmar e, se necessário, corrigir a visibilidade do repositório GitHub (item 5.1).
 2. Mover a validação de permissão (admin/limitado) para o Worker, eliminando a dependência de dados do `localStorage` para autorização.
 3. Substituir o hash de senha por um algoritmo com *salt* e custo computacional.
 4. Restringir o campo `path` da rota `/api/sync` a uma lista fixa de arquivos.
 5. Definir `ALLOWED_ORIGIN` explicitamente em todos os ambientes e adicionar limite de tentativas ao Worker.
 6. Validar o funcionamento real do endpoint HTTP do Neon em produção.
-

8. Histórico de versões (resumo)

2.0.0 (jun/2026) — proxy via Cloudflare Worker, sessão em `sessionStorage`, timeout de inatividade, hash SHA-256, recuperação de senha, log de atividades, busca em tempo real, onboarding, página de perfil, política de privacidade, toasts, responsividade mobile.

1.0.0 (mai/2025) — versão inicial com login e validação via `perfis.json`, módulos de Contas a Receber/Pagar, Fluxo de Caixa e DRE, gerenciamento de clientes com sincronização direta via token do GitHub.